

AI-Based Inventory Management System

Full Module-Wise System Architecture & Technical Manual

Document Version: 2.0 (Comprehensive)	Target Environment: Node.js + Python AI + MySQL
Author / System Architect: Lead AI Software Engineer	Date: August 2026
Project Scope: Enterprise Multi-Tenant Inventory & POS	Status: Production Ready

1. Executive Summary & Architecture Overview

The **AI-Based Inventory Management System** is an end-to-end, enterprise-grade web platform designed to streamline stock control, automate demand forecasting, manage suppliers and purchase orders, process sales via an interactive Point of Sale (POS) interface, and track product expiry dates. By integrating a dedicated Python-powered AI Machine Learning service with a Node.js Express API and a React dynamic frontend, the system minimizes stockouts, eliminates manual inventory tracking errors, and optimizes inventory holding costs using statistical safety stock formulas.

Layer	Technology Stack	Key Purpose & Components
Frontend UI	React 18, Vite, TailwindCSS, Lucide Icons, HTML5 Canvas	Responsive SPA, POS Cashier desk, Barcode/QR generator, Real-time charts
Backend API	Node.js, Express.js, mysql2/promise, JWT, bcrypt	REST API endpoints, MySQL Connection Pooling, Transactions, Auth middleware
AI Microservice	Python 3, Flask, scikit-learn, pandas, numpy	Demand forecasting (RandomForestRegressor + LinearTrend), ROP & Safety Stock calculations
Database	MySQL 8.0 / MariaDB	Relational schema with 12 normalized tables, Foreign keys, Cascade & Null constraints
Infrastructure	Docker, Docker Compose, Nginx Reverse Proxy	Containerized multi-service deployment, environment configurations

2. Database Schema & Data Dictionary

The relational database schema consists of **12 normalized tables** engineered for data integrity, fast transactional queries, and multi-tenant data isolation via ``owner_id`` scoping.

Table Name	Fields & Data Constraints
1. Users Table	<code>`id`</code> (INT PK), <code>`name`</code> (VARCHAR), <code>`email`</code> (VARCHAR UNIQUE), <code>`password_hash`</code> (VARCHAR), <code>`role`</code> (VARCHAR: Admin/Owner/Manager/Staff), <code>`phone`</code> (VARCHAR), <code>`permissions`</code> (JSON), <code>`owner_id`</code> (INT FK), <code>`reset_token`</code> (VARCHAR), <code>`reset_token_expires`</code> (DATETIME)

2. Categories Table	`id` (INT PK), `category_name` (VARCHAR UNIQUE), `description` (TEXT), `created_at` (TIMESTAMP)
3. Suppliers Table	`id` (INT PK), `name` (VARCHAR), `phone` (VARCHAR), `email` (VARCHAR), `address` (TEXT), `owner_id` (INT FK)
4. Products Table	`id` (INT PK), `name` (VARCHAR), `category_id` (INT FK), `cost_price` (DECIMAL), `selling_price` (DECIMAL), `quantity` (INT), `reorder_level` (INT), `barcode` (VARCHAR UNIQUE), `qr_code` (VARCHAR), `supplier_id` (INT FK), `expiry_date` (DATE), `owner_id` (INT FK)
5. Customers Table	`id` (INT PK), `name` (VARCHAR), `phone` (VARCHAR UNIQUE), `email` (VARCHAR), `address` (TEXT), `owner_id` (INT FK)
6. Purchases Table	`id` (INT PK), `supplier_id` (INT FK), `user_id` (INT FK), `total_amount` (DECIMAL), `status` (ENUM: Completed/Pending/Cancelled), `notes` (TEXT), `owner_id` (INT FK)
7. PurchaseDetails Table	`id` (INT PK), `purchase_id` (INT FK CASCADE), `product_id` (INT FK CASCADE), `quantity` (INT), `unit_cost` (DECIMAL)
8. Sales Table	`id` (INT PK), `customer_id` (INT FK SET NULL), `customer_name` (VARCHAR), `user_id` (INT FK), `subtotal` (DECIMAL), `discount` (DECIMAL), `tax_gst` (DECIMAL), `total_amount` (DECIMAL), `payment_method` (ENUM: Cash/Card/UPI/NetBanking), `owner_id` (INT FK)
9. SalesDetails Table	`id` (INT PK), `sale_id` (INT FK CASCADE), `product_id` (INT FK CASCADE), `quantity` (INT), `unit_price` (DECIMAL), `total_price` (DECIMAL)
10. InventoryLogs Table	`id` (INT PK), `product_id` (INT FK), `user_id` (INT FK), `action_type` (ENUM: STOCK_IN, STOCK_OUT, SALE, PURCHASE, ADJUSTMENT, EXPIRED_REMOVAL), `quantity_change` (INT), `notes` (TEXT), `owner_id` (INT FK)
11. AIForecast Table	`id` (INT PK), `product_id` (INT FK), `forecast_date` (DATE), `predicted_demand` (INT), `confidence_score` (DECIMAL), `reorder_recommended_qty` (INT)
12. RolePermissions Table	`id` (INT PK), `role` (VARCHAR UNIQUE), `permissions` (JSON), `updated_at` (TIMESTAMP)

3. Detailed Module-Wise Application Specifications

Module 1: User Authentication & Role-Based Access Control (RBAC)

- **Overview:** Secures application access through JSON Web Token (JWT) authentication, password hashing with bcrypt, and dynamic permission resolution.
- **Key Features:** User login with email/password, JWT generation with 24-hour expiration, stored custom permissions JSON per user, role hierarchy (Admin, Owner, Manager, Staff).
- **Technical File References:**
[authController.js](file:///Users/jenis/Documents/mscit/backend/controllers/authController.js),
[authMiddleware.js](file:///Users/jenis/Documents/mscit/backend/middleware/authMiddleware.js),
[Login.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/Login.jsx).
- **Permission Matrix:** Admin has full system bypass; Owner manages staff; Manager handles inventory & sales; Staff is restricted based on specific permissions array.

Module 2: Dashboard & Real-Time Business Analytics Hub

- **Overview:** Central operational control center displaying real-time Key Performance Indicators (KPIs), urgent stock alerts, and financial metrics.
- **Key Features:** Stat Cards (Total Stock Value, Low Stock Count, Near-Expiry Items, Today's Sales), Recent Sales Activity Feed, Low Stock Warnings, Quick Navigation Action buttons.
- **Technical File References:**
[Dashboard.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/Dashboard.jsx),
[reportController.js](file:///Users/jenis/Documents/mscit/backend/controllers/reportController.js).
- **Data Aggregation:** Queries database using MySQL SUM/COUNT aggregations filtered by current logged-in `owner\_id`.

Module 3: Products Catalog & Category Management

- **Overview:** End-to-end inventory item management featuring pricing, category classification, reorder threshold configuration, and barcode generation.
- **Key Features:** Create/Edit/Delete products, Category mapping, Cost vs Selling Price markup analysis, Image URL link attachment, EAN-13 style Barcode & QR code auto-generation, Filter by low-stock and category.
- **Technical File References:** [Products.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/Products.jsx),
[productController.js](file:///Users/jenis/Documents/mscit/backend/controllers/productController.js).
- **Barcode Generation:** Auto-generates unique 12-digit EAN barcode starting with prefix '890' if not provided.

Module 4: Point of Sale (POS) & Automated Billing Desk

- **Overview:** Interactive, high-speed cashier checkout page designed for rapid retail transaction processing.
- **Key Features:** Live search & instant Barcode scanner input listener, Real-time stock availability validation, Quantity incremental adjustment, Customer phone lookup/auto-creation, Multi-payment methods (Cash, Card, UPI, NetBanking), Discount & Tax/GST calculation, Instant printable Invoice Modal.

- **Technical File References:** [POS.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/POS.jsx), [salesController.js](file:///Users/jenis/Documents/mscit/backend/controllers/salesController.js), [InvoiceModal.jsx](file:///Users/jenis/Documents/mscit/frontend/src/components/InvoiceModal.jsx).
- **ACID Transaction Guarantee:** Uses MySQL database transactions (`beginTransaction`, `commit`, `rollback`) to ensure stock reduction and sale entry execute atomically.

Module 5: Supplier Management & Purchase Procurement System

- **Overview:** Manages vendor relationships, contact info, and stock replenishment purchase orders.
- **Key Features:** Supplier directory (name, phone, email, address), Create Purchase Order with multiple item lines, Automatic stock increment upon purchase completion, Cost price updating, Purchase log creation.
- **Technical File References:** [Suppliers.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/Suppliers.jsx), [Purchases.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/Purchases.jsx), [purchaseController.js](file:///Users/jenis/Documents/mscit/backend/controllers/purchaseController.js).
- **Stock Replenishment Flow:** Creating a completed purchase automatically adds quantity to the target product and writes a `STOCK\_IN` inventory log.

Module 6: Inventory Movement Audit & Stock Adjustment System

- **Overview:** Complete audit trail tracking every single item movement across the entire application lifecycle.
- **Key Features:** Immutable log repository (`InventoryLogs` table), Log action types (`STOCK\_IN`, `STOCK\_OUT`, `SALE`, `PURCHASE`, `ADJUSTMENT`, `EXPIRED\_REMOVAL`), Manual stock adjustment modal with audit notes, User accountability tracking.
- **Technical File References:** [InventoryAudit.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/InventoryAudit.jsx), [inventoryController.js](file:///Users/jenis/Documents/mscit/backend/controllers/inventoryController.js).
- **Compliance:** Ensures complete auditability for shrinkage, damage, discrepancies, and audit reviews.

Module 7: Expiry Tracking & Automated Waste Mitigation

- **Overview:** Monitors perishable products and alerts management before stock expires to prevent financial losses.
- **Key Features:** Expiry date tracking per product batch, Color-coded urgency badges (Expired = Red, Near Expiry < 60 days = Amber, Fresh = Green), One-click Write-Off / Removal action that removes stock and logs `EXPIRED\_REMOVAL`.
- **Technical File References:** [ExpiryAlerts.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/ExpiryAlerts.jsx), [productController.js](file:///Users/jenis/Documents/mscit/backend/controllers/productController.js).
- **Loss Prevention:** Enables promotional clearance discounts on items approaching 30 days to expiration.

Module 8: AI Demand Forecasting Engine

- **Overview:** Python-driven machine learning engine that forecasts 30-day future demand based on historical sales time-series data.

- **Key Features:** `RandomForestRegressor` model trained on day index, day of week, day of month, month, and weekend indicator features; `LinearRegression` slope calculation for trend classification (Fast-Growing, Steady, Declining); Confidence score estimation.
- **Technical File References:**
[AIDemandForecast.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/AIDemandForecast.jsx),
[app.py](file:///Users/jenis/Documents/mscit/ai/app.py),
[model.py](file:///Users/jenis/Documents/mscit/ai/model.py).
- **Heuristic Fallback:** Automatically applies a statistical moving average for new products with fewer than 3 historical sales records.

Module 9: AI Smart Reorder Hub & Inventory Optimization

- **Overview:** Computes optimal stock reorder points (ROP) and recommended order quantities using statistical inventory control equations.
- **Key Features:** Automatic calculation of Safety Stock at 95% service level ($SS = 1.65 \times \sqrt{L} \times \sigma_d$), Reorder Point ($ROP = (d \times L) + SS$), Urgency ranking (CRITICAL, HIGH, MEDIUM, LOW), One-click auto-generated purchase order pre-filling target supplier.
- **Technical File References:**
[AIReorderHub.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/AIReorderHub.jsx),
[model.py](file:///Users/jenis/Documents/mscit/ai/model.py).
- **Automation:** Eliminates guesswork by generating precise purchase orders based on AI-calculated lead time and forecasted demand.

Module 10: Advanced Reports & Financial Analytics Engine

- **Overview:** Comprehensive business intelligence reporting module delivering detailed sales, profit, and stock health analytics.
- **Key Features:** Date range filtering, Revenue vs Cost vs Net Profit calculations, Top-selling products ranking, Category revenue distribution breakdown, Export reports to CSV & printable summary layout.
- **Technical File References:** [Reports.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/Reports.jsx), [reportController.js](file:///Users/jenis/Documents/mscit/backend/controllers/reportController.js).
- **Profit Insights:** Calculates gross margin percentage per product line and overall store profitability.

Module 11: Multi-Tenant Hierarchy & User Management

- **Overview:** Enterprise tenant management enabling Store Owners to provision staff accounts with granular custom permissions.
- **Key Features:** Create/Edit/Disable staff user accounts, Role assignment (Manager, Cashier, Stock Clerk), Custom JSON permissions toggle (e.g. `pos\_access`, `manage\_products`, `view\_reports`, `manage\_users`), Automatic `owner\_id` scoping across all DB tables.
- **Technical File References:**
[UserManagement.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/UserManagement.jsx),
[userController.js](file:///Users/jenis/Documents/mscit/backend/controllers/userController.js).
- **Tenant Security:** All queries enforce `WHERE owner\_id = ?` preventing cross-tenant data leakage.

Module 12: Barcode & QR Code Utility Engine

- **Overview:** Visual rendering component for generating, displaying, and printing physical barcode and QR code stickers for stock labeling.
- **Key Features:** Dynamic HTML5 Canvas rendering of EAN-13 barcodes, QR code generation, Modal print preview with custom sticker layout, High-resolution SVG export.
- **Technical File References:**
[BarcodeModal.jsx](file:///Users/jenis/Documents/mscit/frontend/src/components/BarcodeModal.jsx),
[Products.jsx](file:///Users/jenis/Documents/mscit/frontend/src/pages/Products.jsx).
- **Hardware Compatibility:** Compatible with standard thermal label printers and handheld laser scanners.

4. AI Engine Algorithms & Inventory Control Formulae

The AI Engine operates as a microservice running Flask on port 5001. It evaluates historical sales velocity and calculates reorder metrics using statistical inventory theory.

Metric	Mathematical Formula / Logic	Description & Variables
Average Daily Demand (d)	$d = D_{30} / 30$	Calculates predicted daily consumption from 30-day forecasted demand (D_{30}).
Safety Stock (SS)	$SS = Z * \text{sqrt}(L) * \text{sigma}_d$ $SS = 1.65 * \text{sqrt}(L) * (d * 0.3)$	Buffers against demand volatility. Uses $Z = 1.65$ for 95% service level, lead time L (days), and 30% std dev estimation.
Reorder Point (ROP)	$ROP = (d * L) + SS$	Threshold stock level that triggers a replenishment purchase order before stockout occurs.
Recommended Order Qty	$Q_{rec} = \text{max}(0, (D_{30} + SS) - S_{curr})$	Target inventory level minus current stock (S_{curr}) to cover 30 days of sales plus safety stock.
Demand Forecasting	RandomForestRegressor(n_estimators=100) Features: [day_index, day_of_week, day_of_month, month, is_weekend]	Ensemble regression tree captures weekly patterns, weekend surges, and monthly seasonality.
Anomaly Detection	if quantity_change >= 20 and action in ['SALE', 'STOCK_OUT', 'ADJUSTMENT']	Flags sudden large inventory drops to prevent unauthorized theft or unrecorded bulk losses.

5. Complete REST API Endpoint Specification

Method	Endpoint Path	Module	Description
POST	/api/auth/login	Auth	Authenticates user credentials, returns JWT token & user object.
POST	/api/auth/register	Auth	Registers new Store Owner or admin account.
POST	/api/auth/forgot-password	Auth	Generates 6-digit OTP reset token with 15-min expiration for user account.
POST	/api/auth/reset-password	Auth	Validates OTP reset token and updates password hash securely.
GET	/api/products	Products	Fetches list of products with optional filters (?search=&low;_stock=true &near;_expiry=true).
POST	/api/products	Products	Creates a new product record; auto-generates barcode/QR if empty.
PUT	/api/products/:id	Products	Updates existing product details, price, or stock levels.
DELETE	/api/products/:id	Products	Deletes a product record (Owner/Admin only).
POST	/api/sales	Sales	Executes POS sale transaction, updates stock, creates SalesDetails & InventoryLog.
GET	/api/sales	Sales	Retrieves historical sales transaction records with pagination.
GET	/api/suppliers	Suppliers	Fetches all active vendor/supplier contact profiles.

POST	/api/purchases	Purchases	Creates stock replenishment purchase order and auto-increments product stock.
GET	/api/inventory/logs	Inventory	Fetches complete immutable movement audit log trail.
POST	/api/inventory/adjust	Inventory	Executes manual stock adjustment with required audit reason notes.
GET	/api/reports/dashboard	Reports	Aggregates key financial & inventory metrics for dashboard widgets.
POST	/api/ai/forecast	AI Engine	Calculates 30-day machine learning demand forecast for a given product.
POST	/api/ai/reorder	AI Engine	Computes statistical Reorder Point & Safety Stock recommendation.
POST	/api/ai/batch-analysis	AI Engine	Batch processes all products to generate reorder alerts and demand trends.
GET	/api/users	Users	Lists all staff accounts under current logged-in Store Owner.
POST	/api/users	Users	Creates new staff account with custom JSON permissions array.

6. Installation, Configuration & System Setup Guide

Step 1: Database Setup

1. Create MySQL Database: `CREATE DATABASE inventory_db;`
2. Execute SQL schema script: `mysql -u root -p inventory_db < database/schema.sql`
3. (Optional) Insert seed data: `mysql -u root -p inventory_db < database/seed.sql`

Step 2: Backend Node.js Service Setup

1. Navigate to backend/ directory and install packages: `npm install`
2. Configure environment file backend/.env:
PORT=5000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=yourpassword
DB_NAME=inventory_db
JWT_SECRET=your_super_secret_jwt_key
3. Launch server: `npm run dev` (Server runs on `http://localhost:5000`)

Step 3: AI Engine Python Service Setup

1. Navigate to ai/ directory.
2. Create virtual environment & install requirements: `pip install -r requirements.txt`
3. Start Flask AI server: `python app.py` (Runs on `http://localhost:5001`)

Step 4: Frontend React Web Client Setup

1. Navigate to frontend/ directory.
2. Install npm dependencies: `npm install`
3. Launch Vite development server: `npm run dev` (Client runs on `http://localhost:5173`)

Step 5: Docker Multi-Container Deployment

Build and launch all services simultaneously using Docker Compose:

```
docker-compose up --build -d
```